Section: 14. Functions

Lecture: 158. Iterators and Generators

Section 1: Lecture Summary

This lecture explains the concepts of **iterators** and **generators** in Python, both of which are mechanisms to traverse or generate sequences of data. It begins by illustrating how a for loop iterates over a list and then reveals the internal working of this iteration using Python's built-in **iter()** function to create an iterator and **next()** to retrieve successive elements one by one. The lecture covers how to manually control iteration with these functions and demonstrates the behavior of iterators on different iterable types such as tuples, sets, dictionaries, strings, and ranges.

The focus then shifts to **generators**, special functions that produce a sequence of values on-the-fly without storing the entire sequence in memory. The lecture shows how Python's built-in `range()` function acts like a generator and explains how to write custom generator functions using the **yield** keyword instead of return. A sample implementation of a custom range-like generator function is provided. Finally, an exercise is posed to write a generator that cycles repeatedly through the days of the week indefinitely.

Section 2: Key Concepts and Explanations

Iterators

- An iterator is an object that enables traversing through all the elements of an iterable one at a time.

- Python provides the **iter()** built-in function that takes an iterable (like a list, tuple, set, string, dictionary, range) and returns an iterator object.

- The **next()** built-in function retrieves the next item from the iterator and advances the internal pointer.

- When the iterator reaches the end, a **StopIteration** exception is raised, which signals the end of iteration. The for loop internally handles this exception.

- Example iterables:

  - List, tuple, string: ordered sequences that return elements in order.

  - Set: unordered collection; iteration order is not guaranteed.

  - Dictionary: iterating returns keys by default.

- Range: generates numbers in a range, behaves like an iterable object.

Generators

- A generator is a special kind of iterator defined by a function that uses the **yield** keyword.

- Unlike regular functions that return a value and exit, a generator yields a value and pauses its state, resuming on the next call.

- Generators produce items lazily (on demand) instead of building and returning a complete list up front.

- The built-in `range()` function behaves like a generator: it produces each number on demand indirectly.

- Syntax for generators:

  - Use `yield` instead of `return` inside a function.

  - Once the function hits `yield`, it outputs the value but retains its local state for next call.

- Stopping condition: when the function finishes or no more yields, a StopIteration is raised.

- Circular or infinite generators can be implemented with `while True` loops and cycling indices.

Section 3: Example Code and Use Cases

Iterators on a list

The next call would raise StopIteration

```
L1 = [5, 6, 10, 11, 15]
it = iter(L1)
print(next(it))  # Output: 5
print(next(it))  # Output: 6
print(next(it))  # Output: 10
print(next(it))  # Output: 11
print(next(it))  # Output: 15
```

Explanation: The `iter()` function returns an iterator on the list. `next()` retrieves each element sequentially until the list is exhausted.

Iterators on other iterable types

Tuple

Set (unordered)

Dictionary

String

Range

```
T1 = (5, 6, 10)
it = iter(T1)
print(next(it))  # 5

S1 = {7, 4, 9}
it = iter(S1)
print(next(it))  # Unordered element, could be any of 7,4,9

D1 = {'a': 1, 'b': 2}
it = iter(D1)
print(next(it))  # 'a' (key of the dictionary)

s = "Hello"
it = iter(s)
print(next(it))  # 'H'

r = range(3, 10)
it = iter(r)
print(next(it))  # 3
```

Explanation: Iterators work across different iterable types, returning elements according to their ordering (if any).

Custom Generator Function Mimicking range()

Another next(m) call raises StopIteration

```
def myrange(n):
    i = 0
    while i < n:
        yield i
        i += 1

m = myrange(4)
print(next(m))  # 0
print(next(m))  # 1
print(next(m))  # 2
print(next(m))  # 3
```

Explanation: This function uses `yield` to return values lazily one by one until `i` reaches `n`.

Generator function cycling through days of the week infinitely

Keeps cycling through days indefinitely

```
def days():
    d = ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat']
    i = 0
    while True:
        yield d[i]
        i = (i + 1) % 7

day_generator = days()
print(next(day_generator))  # Sun
print(next(day_generator))  # Mon
print(next(day_generator))  # Tue
```

Explanation: Continuous `while True` loop with a modulo index cycles endlessly through the list of week days.

Section 4: Key Takeaways

- **Iterator** allows controlled traversal of iterable objects using `iter()` and `next()` functions.

- All iterable Python objects (list, tuple, set, dict, string, range) can generate iterators.

- When an iterator runs out of elements, a **StopIteration** exception is raised.

- **Generators** are special iterators created with functions using `yield` for lazy, on-demand value generation.

- `yield` pauses the function state and returns a value, unlike `return` which terminates the function.

- Custom generators enable efficient and memory-friendly production of sequences, including infinite sequences.

- Built-in functions like `range()` behave like generators.

- Understanding iterators and generators is key to working with many Python libraries and writing efficient code.